GOLD-001 — Batch 004
Genuine Multi-Hop & Coverage Expansion

Production RAG v1 · evidence-candidate authoring · 15 candidates proposed, none verified, none gold

STATUS — NOTHING IN BATCH 004 IS GOLD

Every candidate is `candidate_unverified`. No retrieval system was run against any of them, SYSTEM-A and SYSTEM-B remain frozen and unexecuted, and the holdout is not frozen. The confirmed eligible count is still **53**, from batches 001–003; batch 004 adds candidates, not cases.

15 / 20
candidates returned against the target of 20

1 / 559
bridge pairs that passed the composition check

15 / 15
precheck holdout-ready (not human approval)

53
confirmed eligible cases unchanged by this batch

1. Why this batch exists

Batch 003 closed with **zero** genuine multi-hop cases. Four candidates carried the label and none survived review: each drew on two spans, which made the label look earned, while the answer was the two spans' contents rather than anything that followed from combining them. Multi-span is an evidence shape. Multi-hop is a reasoning type. Batch 003 conflated them and had no check that could fail.

Batch 004 runs the composition check **before** export. A pair is rejected when either span already answers the whole question, when the bridge entity is not in both spans, when a hop's assertion is not carried by its own span, when the two spans sit in different providers' documentation, when span 1 merely enumerates values, or when span 2's conditional tests something other than the bridge entity's own state.

2. Authoritative starting state

Read from `GOLD-001-eligibility-status.json` and the closed batch records it names, not from prose summaries.

batch	human_verified	holdout_eligible	rejected	genuine multi-hop
001	16	16	2	0
002	17	17	1	0
003	20	20	0	0
total	**53**	**53**	**3**	**0**

`retrieval_was_not_run = true` on all three closed batches. Holdout frozen: **false**. SYSTEM-A and SYSTEM-B config hashes unchanged.

3. The finding: how often a hop is not a hop

The composer tested **559** bridge pairs drawn from facts that share a specific API symbol. **1** passed the composition check; **558** were rejected.

rejection reason (§30)	pairs	share
the spans were two unrelated lookups	379	68%
no bridge relationship existed	147	26%
span 1 alone answered the full question	16	3%
span 2 alone answered the full question	16	3%
the composed answer introduced unsupported inference	0	0%

Counted from each check's own reason string, one bucket per pair, under the first reason it failed. The report's `unclassified` guard is 0: a check cannot grow a reason the table silently drops.

The same rejections, by the check that made them

check	pairs
span 2 states no condition on the bridge entity	371
span 1 enumerates values rather than stating a requirement about the bridge entity	147
span 1 alone already answers the whole question	16
span 2 alone already answers the whole question	16
the spans are two unrelated lookups	8

> READING THIS HONESTLY
>
> A 1-in-559 yield looks like a failure and is in fact the measurement working. It says that in this corpus two facts sharing an identifier are almost never two halves of an argument. Batch 003 could not produce this number, because it rejected nothing.

4. The one case that passed, in full

GOLD-B004-15 · openai · `multi_document` · 2 documents · composition check **PASS**

Q. If I set `needs_approval` to `True`, what happens?

Bridge entity. `needs_approval` — span 1 states a requirement or constraint on the bridge entity; span 2 states the behaviour that follows

E1 · `ver_ae3bfcc42c733c5051abda30f0f6db07 1327–1436 (109 chars)`

```
Set `needs_approval` to `True` to always require approval or provide an async function that decides per call.
```

E2 · `ver_ae909bf8b4bbbe1d1a11119447f7ac94 16756–16910 (154 chars)`

```
SDK tool approval interruptions are not supported: any function tool whose `needs_approval` setting is not `False` is rejected before the request is sent.
```

Why neither span is enough. Span 1 establishes the condition on needs_approval, but does not state needs_approval. Span 2 states what follows, but does not establish that it applies to needs_approval.

> WHAT A REVIEWER SHOULD CHECK HERE
>
> This case is the batch's whole thesis, and it is the one I would attack first. The composition check is mechanical: it proves that neither span carries the other's critical strings. It cannot prove that span 1 establishes the state span 2's condition tests — that judgement is why this candidate is marked `needs_human_interpretation`.

5. What the batch contains

reasoning type	in batch	target	met	eligible available
`genuine_multi_hop`	1	6–8	MISSED	1
`configuration_interaction`	5	4–5	met	39
`ambiguity_disambiguation`	2	3–4	MISSED	2
`error_behavior`	3	2–3	met	18
`lifecycle_compatibility_migration`	1	2–3	MISSED	1
`exact_lookup`	3	0–3	met	16

The last column is the honest one. Where it equals the batch count, the corpus had nothing more to give under the checks in §6, §9 and §20 — the target was not missed by selection. Where it is far above, the ceiling stopped the batch rather than the material: the reasoning-type ceilings are hard, so a fourth exact lookup cannot fill a multi-hop seat.

provider	in batch	target	met	documents	versions
openai	8	10–12	MISSED	5	6
anthropic	7	8–10	MISSED	5	5

No target was made to read **met** by relabelling a candidate or lowering the evidence standard. §3 of the brief puts quality above count, so the batch came back at 15 rather than padded to 20.

Evidence size

Across 18 spans: mean 141.4, median 144, max 330 characters. 0 over the 1,000-character soft cap, none over the 1,500 hard cap. Multi-hop cases are measured per span, because the size that matters is the size of each anchor.

Removed before export

reason	candidates
fake multi hop	558
reasoning type ceiling	62
not selected diversity	62
duplicate evidence	25
duplicate question	24
failed precheck	4
excluded known failure case	1
blocking anaphora	1

6. The candidates

id	provider	reasoning type	shape	chars	question
01	anthropic	`configuration_interaction`	single_span	209	What happens when using server tools?
02	anthropic	`configuration_interaction`	single_span	180	What happens if Claude calls web search and one of your client tools in the same group of parallel tool calls?
03	anthropic	`error_behavior`	single_span	232	What happens if the most recent assistant message contains `thinking` or `redacted_thinking` blocks that were edited, reordered, filtered out, or reconstructed before being sent back to the API?
04	anthropic	`error_behavior`	single_span	144	What happens if generation then reaches the context window limit?
05	anthropic	`error_behavior`	single_span	128	What happens if Claude attempts more searches than allowed?
06	anthropic	`exact_lookup`	single_span	99	What is the `timezone` option?
07	anthropic	`lifecycle_compatibility_migration`	single_span	156	What happens if you need a hard ceiling on thinking costs?
08	openai	`ambiguity_disambiguation`	multi_span	53	In a `ContentDeltaEvent`, what does the `type` field contain, and how does that differ from `ContentDoneEvent`?
09	openai	`ambiguity_disambiguation`	multi_span	206	In a `FunctionToolCallArgumentsDeltaEvent`, what does the `parsed_arguments` field contain, and how does that differ from `FunctionToolCallArgumentsDoneEvent`?
10	openai	`configuration_interaction`	single_span	142	What happens if you pass a different `RealtimeModel`?
11	openai	`configuration_interaction`	single_span	185	What happens if a tool requires approval?
12	openai	`configuration_interaction`	single_span	330	What happens if you are manually continuing from `result.to_input_list(mode="normalized")`, and `cancel(mode="after_turn")` stops after a tool turn?
13	openai	`exact_lookup`	single_span	121	What is the `input_type` option?
14	openai	`exact_lookup`	single_span	98	What is the `input_filter` option?
15	openai	`genuine_multi_hop`	multi_document	263	If I set `needs_approval` to `True`, what happens?

7. What this document does not say

- No batch-004 candidate is eligible, verified, or gold. `precheck_holdout_ready` is a structural check, not an approval, and only an explicit owner decision can produce `human_verified`.
- No retrieval system was run against any candidate in any batch. No candidate was selected, ordered or worded because of what any system does with it, and no difficulty label in this batch derives from retrieval behaviour.
- The holdout is not frozen, and this document does not freeze it.
- Batches 001–003 are untouched. Their closure hashes are unchanged.

8. Next step, and who owns it

> **THE NEXT STEP IS HUMAN REVIEW, NOT BATCH 005**
>
> 15 candidates need an owner decision before any of them can count. The multi-hop case should be reviewed first and hardest: it is the only one of its kind, and the mechanical check that admitted it cannot judge whether span 1 really establishes the state span 2's condition tests.
>
> Two cases I would flag without being asked. The batch's evidence for `GOLD-B004-04` opens “If generation **then** reaches…”, which leans on a preceding sentence the span does not contain. And 5 bridge pairs passed every other check and were rejected only by the entity-state rule — a reviewer should satisfy themselves that the line was drawn on the evidence rather than to reach a comfortable number. Each is set out in `BATCH-004-near-miss-multihop-review.md`; the review finds every one a correct rejection.

Generated by scripts/build_batch_004_pdf.py from evals/review/gold_review_batch_004.json, GOLD-001-batch-004-generation-report.json, GOLD-001-coverage-status-after-b004-generation.json and GOLD-001-eligibility-status.json. Every count is read from those artifacts at build time. Batch 004, schema 1.2, generated 2026-08-21T05:18:49Z, git commit 2daaa9e62566, corpus snapshot snap_689e336380a054d8039dc35b2c09cd0a, batch_sha256 e7c6d58c3c21b9ece940ae9665cba96a… Raw provider documentation is not redistributed; quoted spans are the short excerpts needed to show the evidence under review.